AWS Level 3 iteration 4 terminal documentation

Claudio Verdi | APL | 24 October 2026

I created an Ubuntu instance with Terraform and used user-data to install Ansible. A local playbook installed Nginx and Git, fetched the website and deployed it automatically. The log and web tests confirmed the result. A second Ansible run made no changes. I then removed the instance and checked its terminated state.

Environment and resources

Region: eu-north-1

Instance: i-0037b3fe4cc3dfff8

Type and operating system: t3.micro / Ubuntu 24.04 LTS

AMI: ami-06fb1802e2b757f09

Subnet: subnet-05d4dca4820d2eb94

Security group: sg-0fb87d0f08a213466

Key pair: devops.school.level3.tabish-admin

Former public IP: 13.62.49.209

AWS provider: 6.70.0

Ansible and ansible-core: 9.2.0 / 2.16.3

Nginx: 1.24.0

Root disk: 8 GiB gp3 / encrypted / delete_on_termination=true

Reference: Level3 Iteration4(3).sh. I adapted package management and file paths to Ubuntu and the website’s actual structure.

Preparation and user-data

I created ~/aws-devops-level3-iteration4 and copied five Terraform files from iteration 3 without state files. I reused the existing security group and SSH key. I checked user-data.web.sh using bash -n.

User-data updates the package index, runs apt-get upgrade and installs Ansible. It then writes /opt/config-and-deploy.web.yml and runs the playbook locally on the EC2 instance.

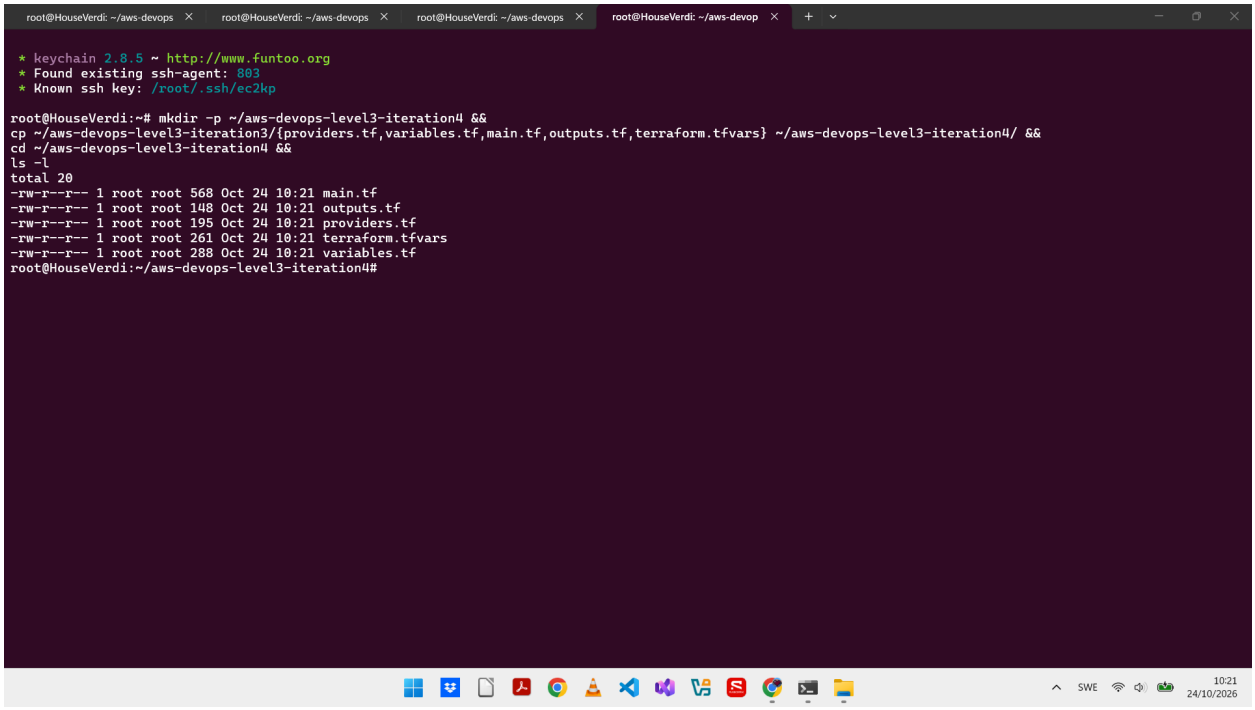


Figure 1. Five Terraform files in the new project directory

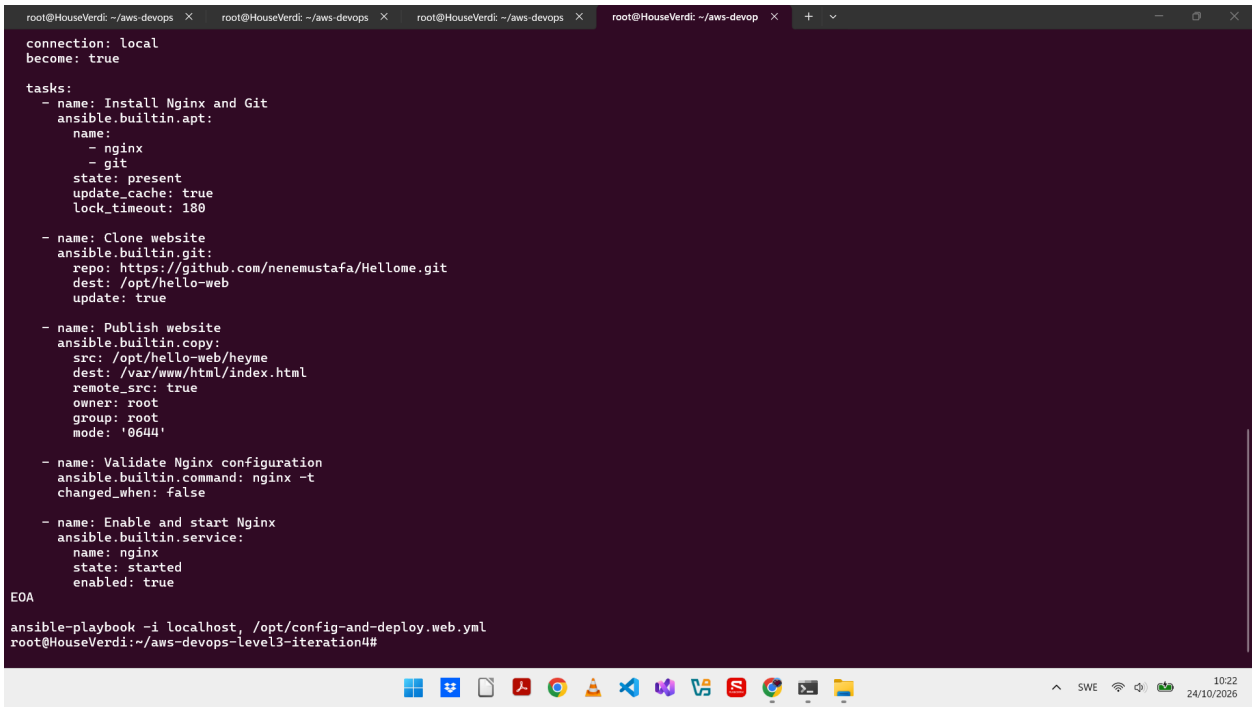


Figure 2. Visible part of the playbook after the syntax check

Terraform validation and planning

terraform fmt, init and validate completed without errors. AWS provider 6.70.0 was installed. main.tf reads user-data from a file, requires IMDSv2 and encrypts the root disk. The plan was saved as tfplan and showed one new resource.

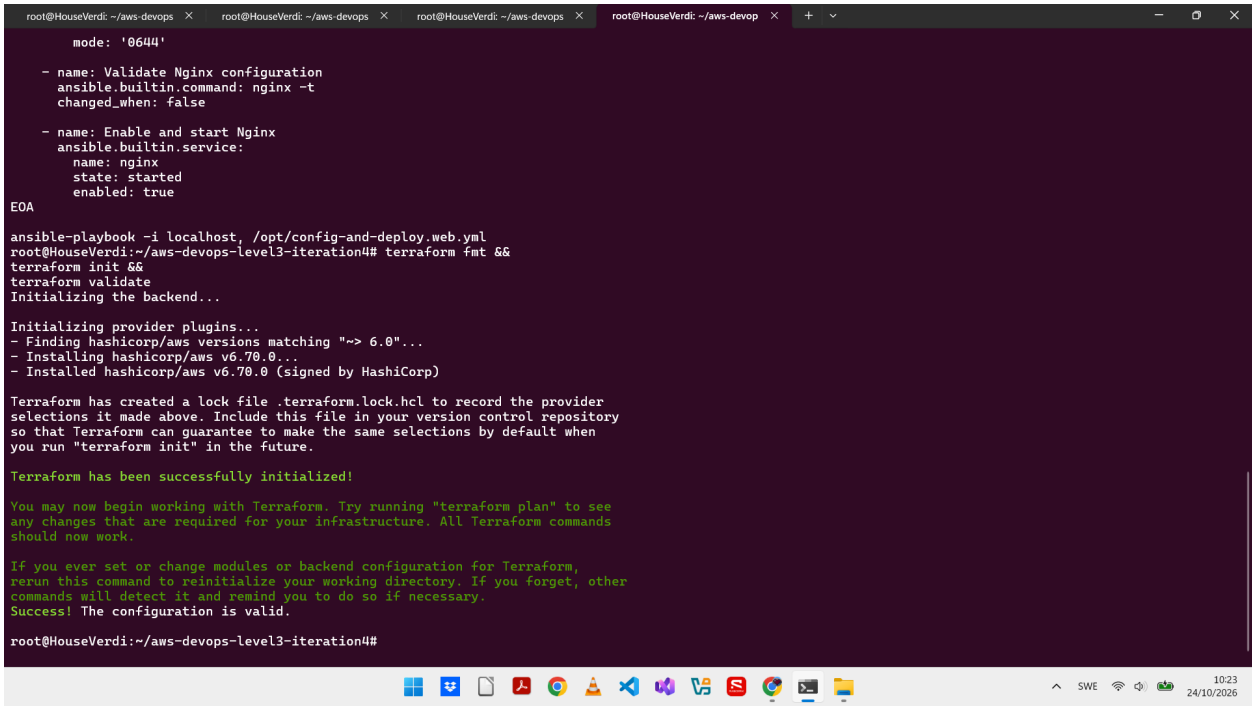


Figure 3. Initialization and successful validation

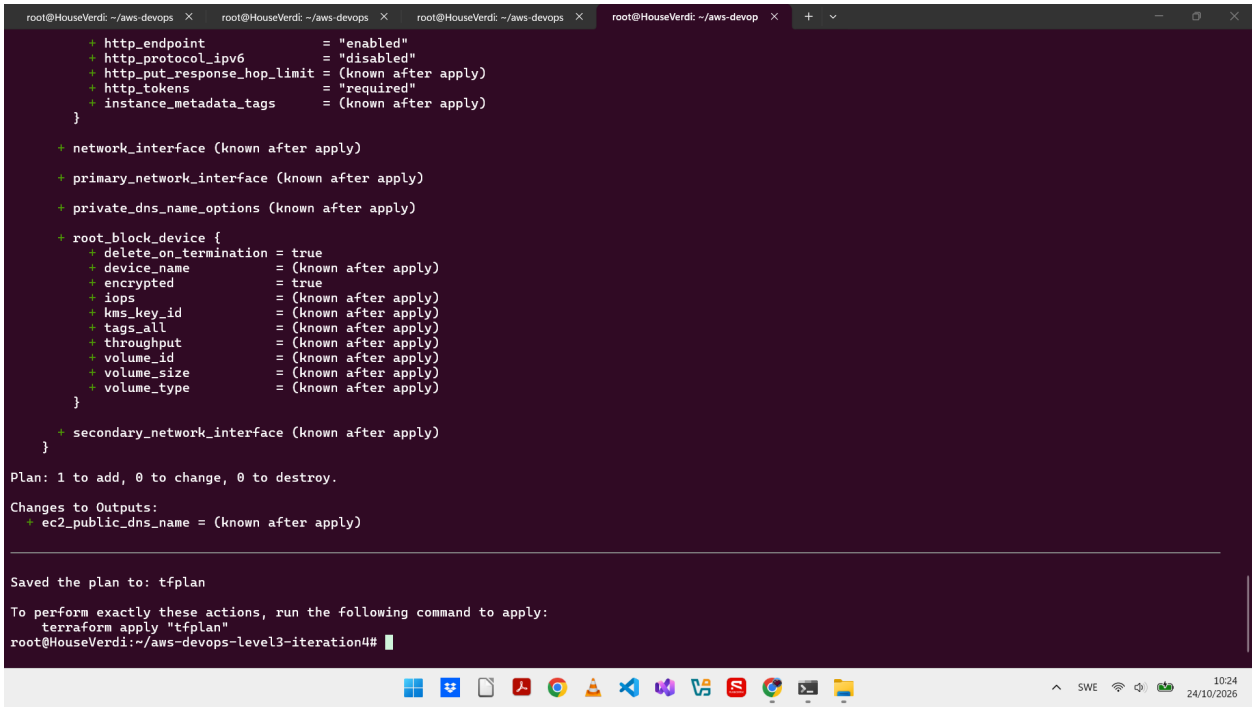


Figure 4. Plan with 1 to add and no other changes

Provisioning and cloud-init

terraform apply tfplan created the instance in 13 seconds. Its DNS name was ec2-13-62-49-209.eu-north-1.compute.amazonaws.com. I read the cloud-init status over SSH as ubuntu using ~/.ssh/id_rsa_level3. The result was status: done.

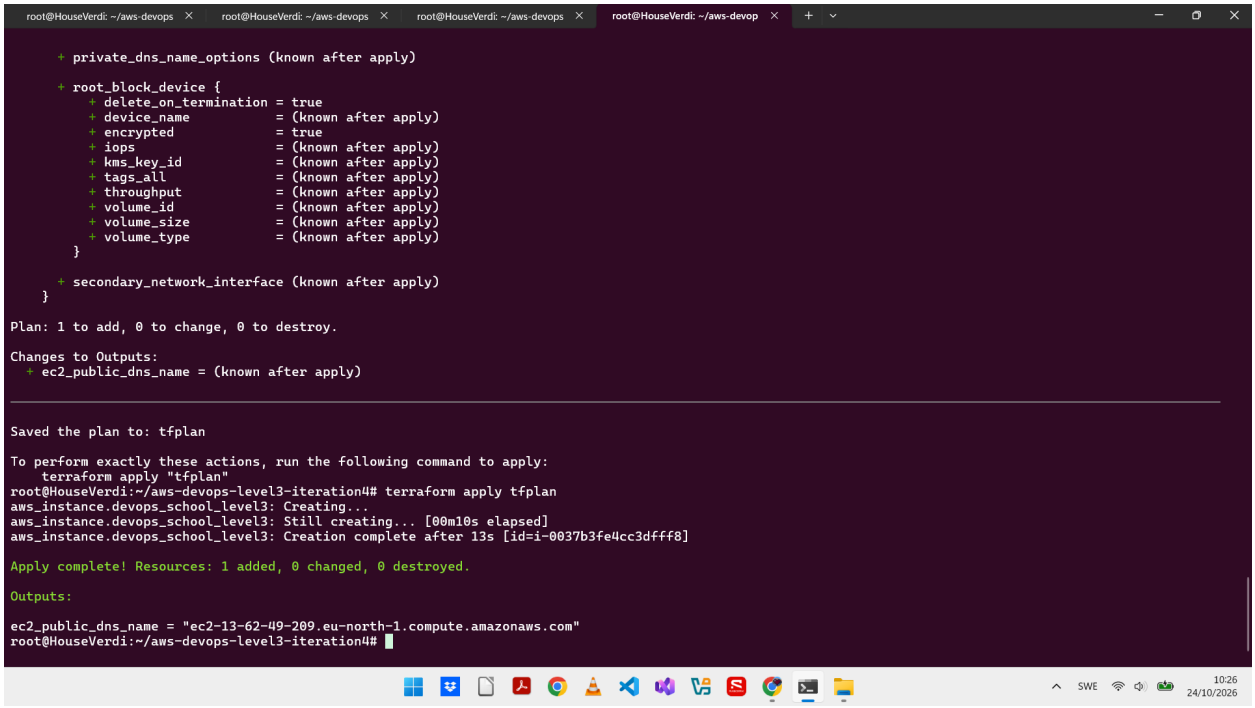


Figure 5. Instance created with ID and DNS name

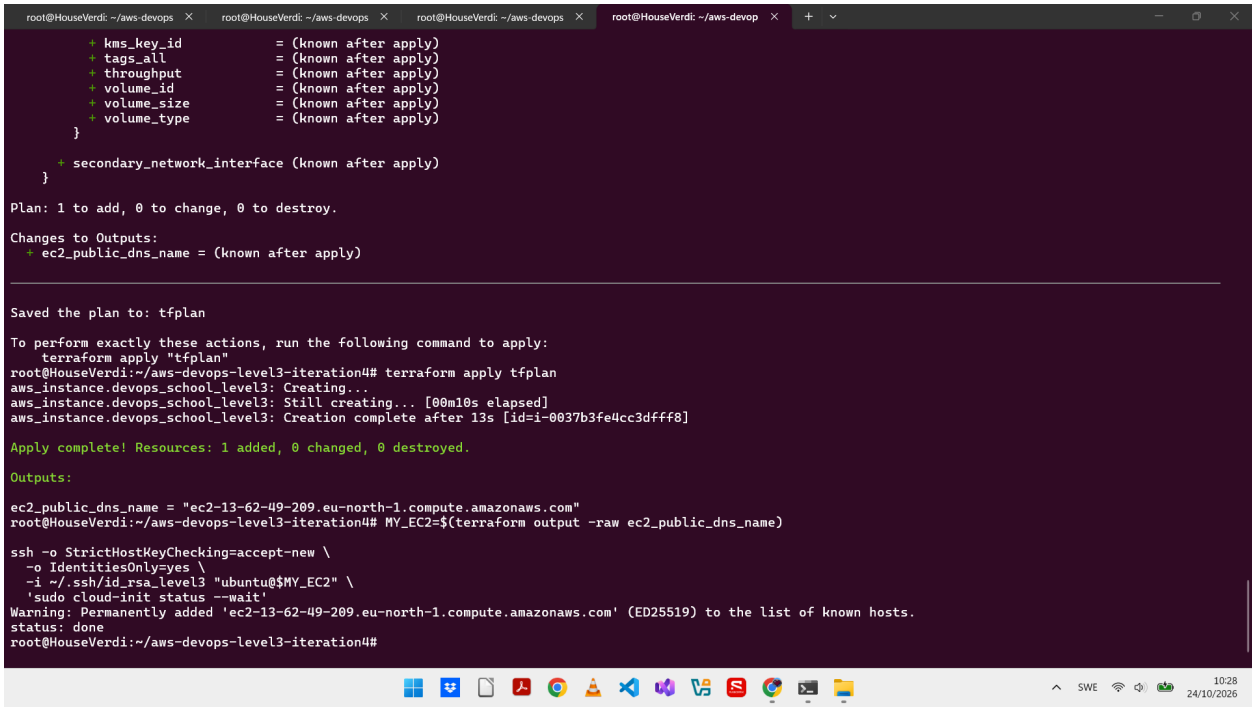


Figure 6. Cloud-init reports status done

Automatic configuration with Ansible

The playbook uses apt for Nginx and Git, git to fetch into /opt/hello-web, and copy with remote_src to deploy heyme as /var/www/html/index.html. It checks nginx -t and ensures Nginx is started and enabled.

PLAY RECAP shows ok=6, changed=3, unreachable=0 and failed=0. The three changes concerned packages, the Git checkout and deployment. Cloud-init finished at 08:27:47 UTC on 24 October 2026, equivalent to 10:27:47 Swedish time, after 96.49 seconds of uptime.

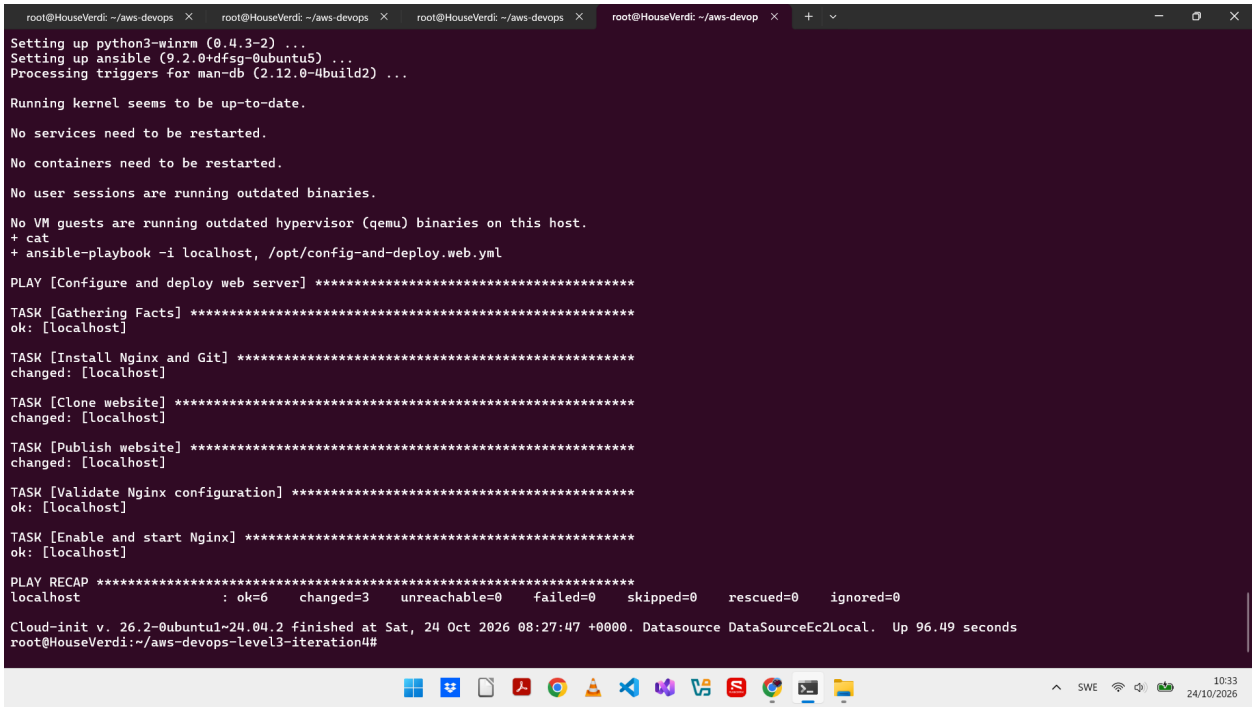


Figure 7. First Ansible run completed without errors

Difference from iteration 3

In iteration 3, installation and deployment used Bash commands. Here, Bash starts a declarative Ansible playbook describing the desired state of packages, files and the service.

HTTP and website content verification

curl -I returned HTTP/1.1 200 OK, nginx/1.24.0 (Ubuntu), Content-Type text/html and Content-Length 499. The content test with the corrected search text Hey me! returned YES and exit code 0.

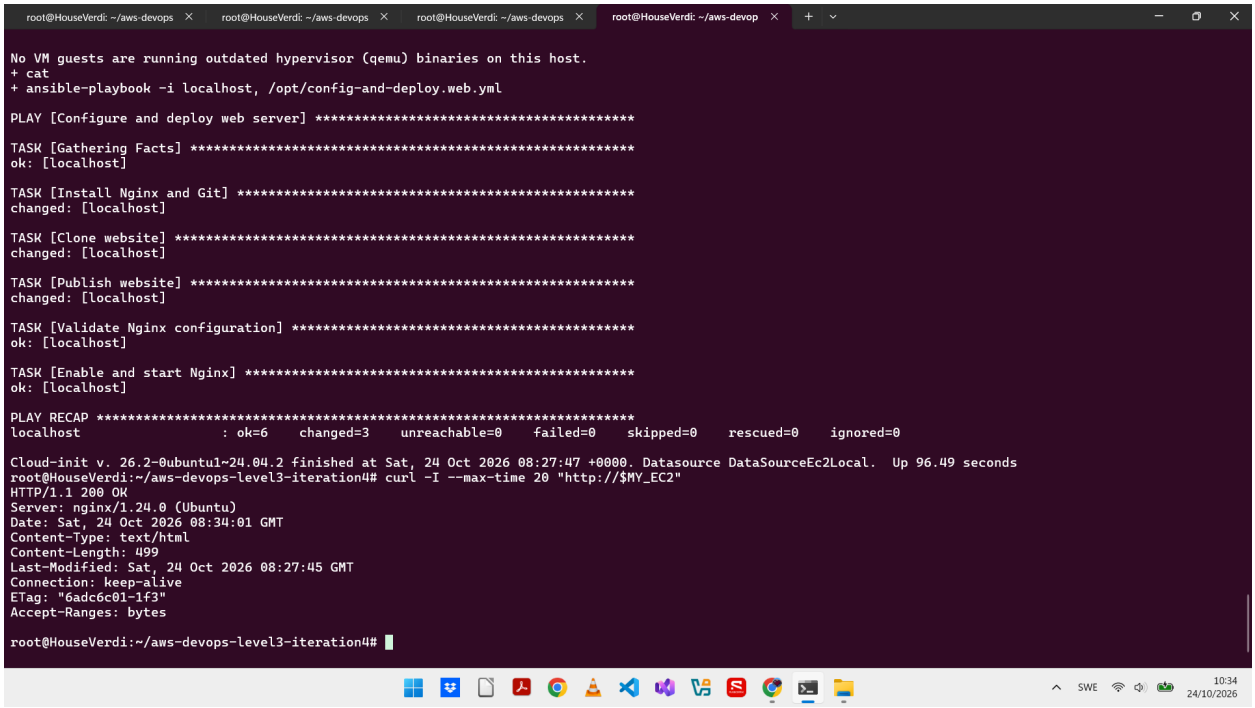


Figure 8. HTTP 200 from the web server

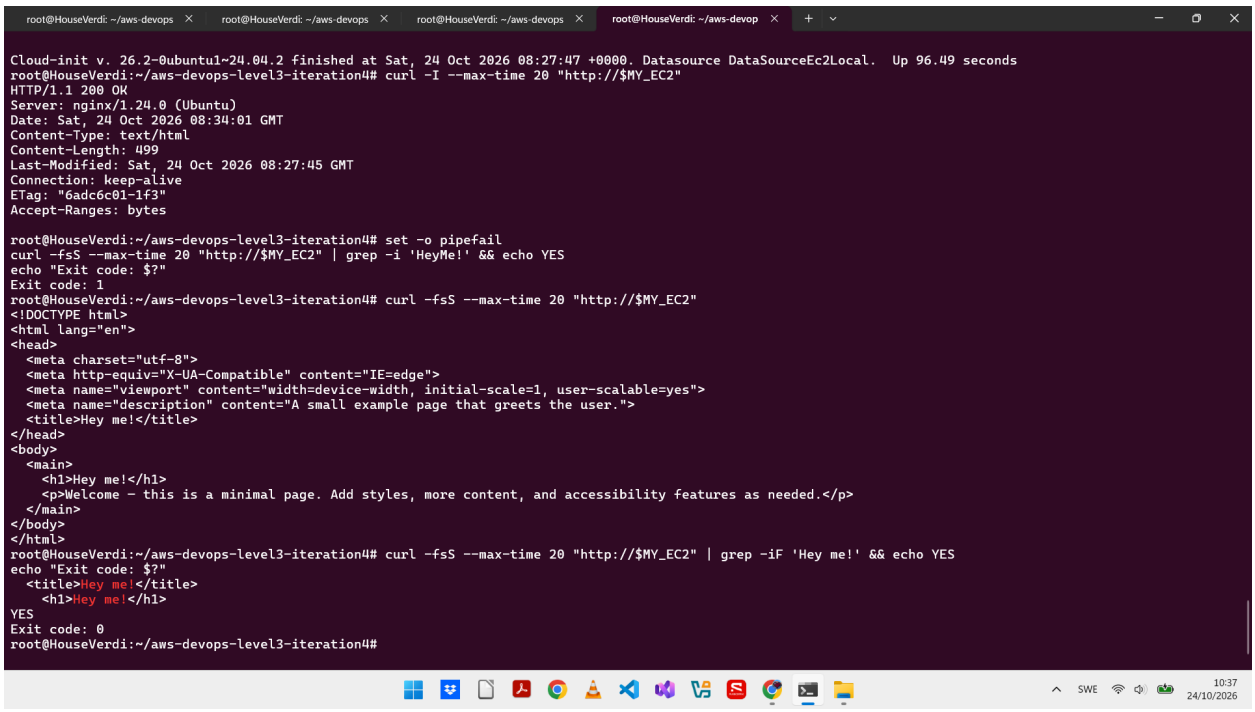


Figure 9. Corrected content test with YES and exit code 0

Browser and infrastructure verification

The browser displayed Hey me! at the instance's DNS address. terraform plan returned No changes. Terraform checks infrastructure; the website content was checked separately.

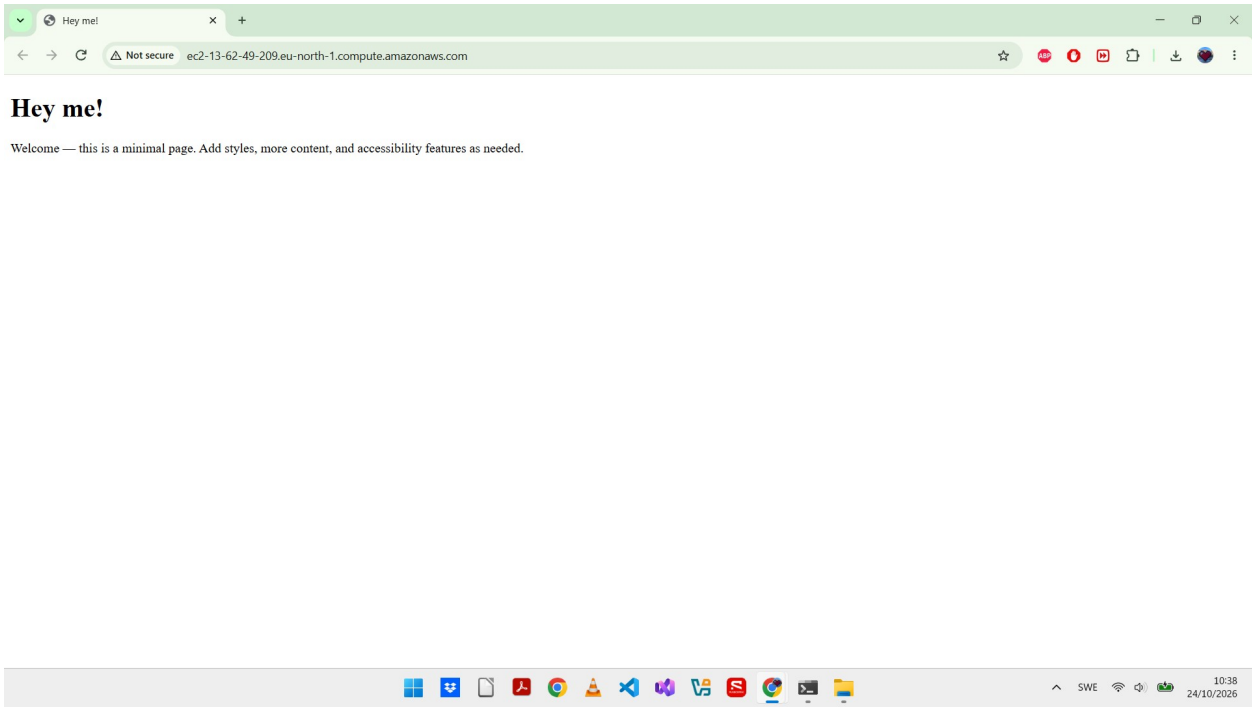


Figure 10. Website at the correct server address

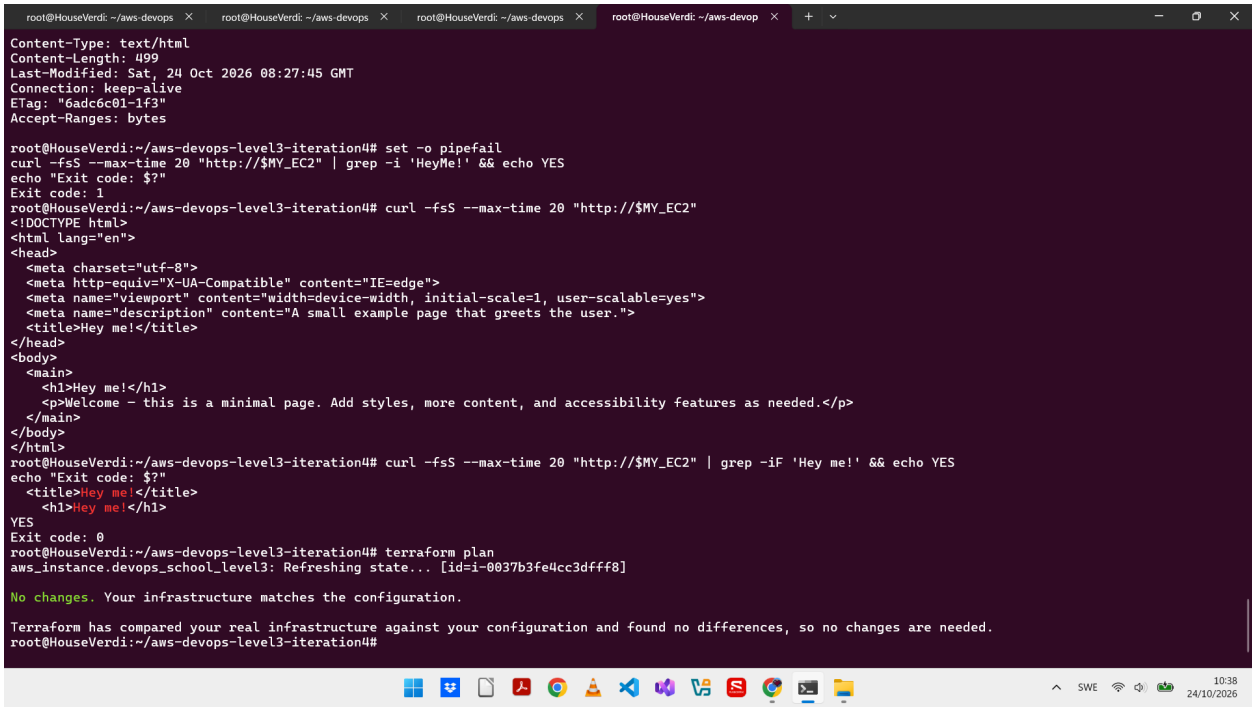


Figure 11. Terraform reports No changes

Second Ansible run

I ran the same playbook again over SSH. All six tasks reported ok and the recap was ok=6, changed=0, unreachable=0, failed=0. This shows that the second run did not need to change the existing state.

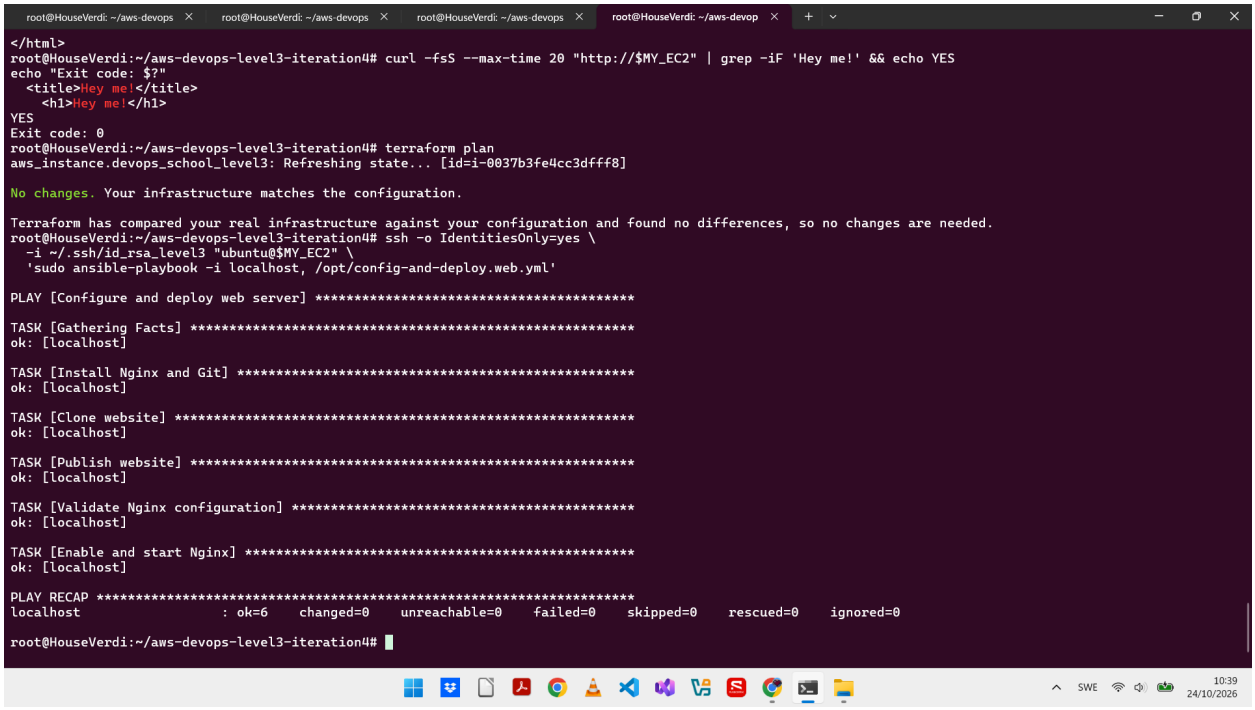


Figure 12. Second run without changes or failures

Interpretation and limitations

The result verifies a repeat run with the packages and Git content available at that time. update=true can fetch later changes from the repository. nginx -t runs each time but reports no change through changed_when: false. No reboot test was performed.

Cleanup and verified removal

I saved the configuration, terminal history and cloud-init log. terraform plan -destroy -out=destroy.tfplan showed one resource to remove. terraform apply destroy.tfplan completed after 33 seconds with 1 destroyed. AWS CLI confirmed terminated on 24 October at 10:44 Swedish time.

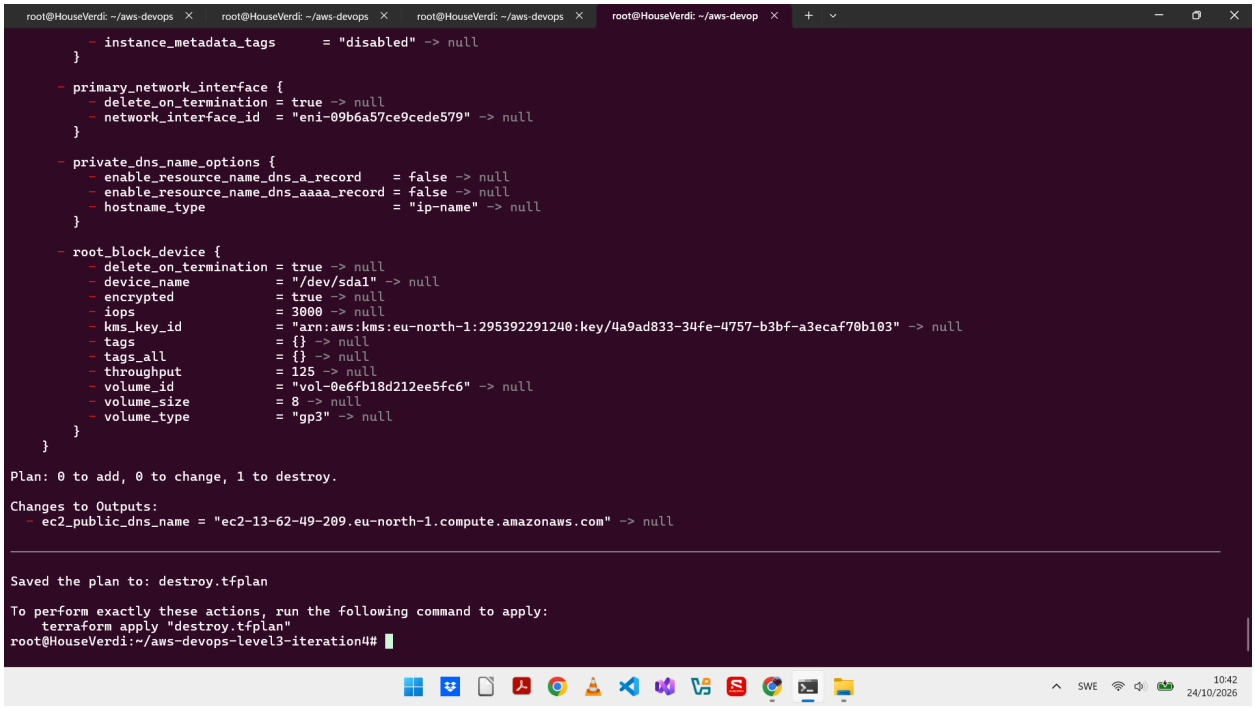


Figure 13. Removal plan and root disk settings

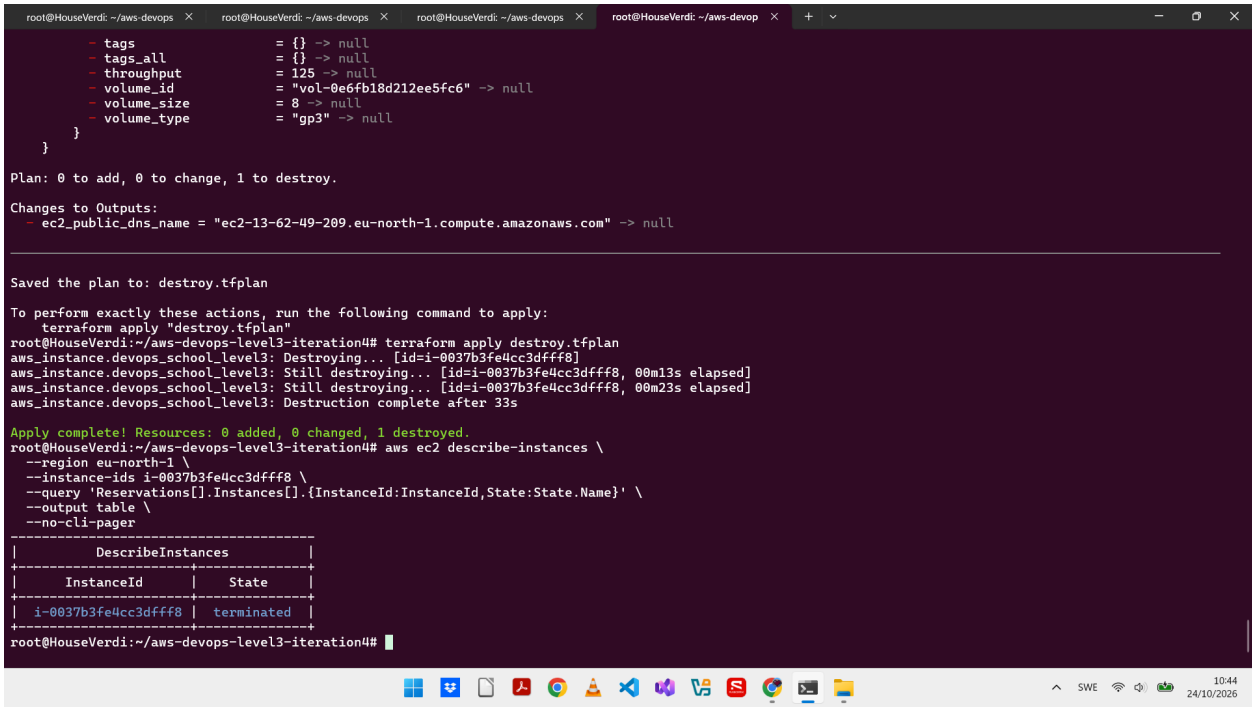


Figure 14. Completed removal and terminated state

Troubleshooting adaptations and conclusion

Content test search text

The first search for HeyMe! returned exit code 1. I read the HTML using curl and found Hey me! with a space. After correcting the search text, the test returned YES and exit code 0. The website did not need changing; the error was in the test's search text.

Adaptations from the reference

I replaced the guide's yum with apt and apt-get for Ubuntu. The SSH user is ubuntu. Instead of synchronize from an html directory, copy with remote_src deploys the file heyme. The target is /var/www/html/index.html. The explicit localhost, inventory and connection: local run the playbook locally on the EC2 instance.

Result and improvements

Terraform, user-data and Ansible produced a working web server with automatic deployment. HTTP, the content test, browser and log confirm the result. A second Ansible run made no changes. Possible improvements include HTTPS and pinning a Git commit for reproducible content.

The root disk had delete_on_termination=true. The existing key pair and security group were managed outside Terraform and were not included in the removal. The report's IP and DNS refer to the period before cleanup.

Evidence

Level3 Iteration4(3).sh; level3-4-konfiguration.txt; updated level3-4-terminal-history(1).txt; level3-4-cloud-init.log and embedded screenshots. Terminal history records commands; screenshots and the server log substantiate their results.

Appendix Terraform base files

providers.tf

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

variables.tf

```
variable "aws_region" {
  type = string
}

variable "ami_id" {
  type = string
}

variable "subnet_id" {
  type = string
}

variable "main_sg_id" {
  type = string
}

variable "ec2_type" {
  type = string
}

variable "ec2_name" {
  type = string
}

variable "keypair" {
  type = string
}
```

Appendix instance and variable values

main.tf

```
resource "aws_instance" "devops_school_level3" {
  ami                         = var.ami_id
  instance_type               = var.ec2_type
  subnet_id                   = var.subnet_id
  key_name                    = var.keypair
  vpc_security_group_ids      = [var.main_sg_id]
  associate_public_ip_address = true

  user_data                   = file("${path.module}/user-data.web.sh")
  user_data_replace_on_change = true

  metadata_options {
    http_tokens = "required"
  }

  root_block_device {
    encrypted = true
  }

  tags = {
    Name = var.ec2_name
  }
}
```

outputs.tf

```
output "ec2_public_dns_name" {
  description = "Public DNS name of the EC2 instance"
  value       = aws_instance.devops_school_level3.public_dns
}
```

terraform.tfvars

```
aws_region = "eu-north-1"
ami_id     = "ami-06fb1802e2b757f09"
subnet_id  = "subnet-05d4dca4820d2eb94"
main_sg_id = "sg-0fb87d0f08a213466"
ec2_type   = "t3.micro"
ec2_name   = "devops-school-level3.tabish-admin"
keypair    = "devops.school.level3.tabish-admin"
```

Appendix user-data and Ansible

```
#!/bin/bash
set -euxo pipefail
export DEBIAN_FRONTEND=noninteractive

apt-get update
apt-get upgrade -y
apt-get install -y ansible

cat > /opt/config-and-deploy.web.yml <<'EOA'
---
- name: Configure and deploy web server
  hosts: localhost
  connection: local
  become: true

  tasks:
    - name: Install Nginx and Git
      ansible.builtin.apt:
        name:
          - nginx
          - git
        state: present
        update_cache: true
        lock_timeout: 180

    - name: Clone website
      ansible.builtin.git:
        repo: https://github.com/nenemustafa/Hellome.git
        dest: /opt/hello-web
        update: true

    - name: Publish website
      ansible.builtin.copy:
        src: /opt/hello-web/heyme
        dest: /var/www/html/index.html
        remote_src: true
        owner: root
        group: root
        mode: '0644'

    - name: Validate Nginx configuration
      ansible.builtin.command: nginx -t
      changed_when: false

    - name: Enable and start Nginx
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true
EOA

ansible-playbook -i localhost, /opt/config-and-deploy.web.yml
```

Appendix verification and cleanup

```
MY_EC2=$(terraform output -raw ec2_public_dns_name)
ssh -o IdentitiesOnly=yes -i ~/.ssh/id_rsa_level3 \
  "ubuntu@$MY_EC2" 'sudo cloud-init status --wait'
ssh -o IdentitiesOnly=yes -i ~/.ssh/id_rsa_level3 \
  "ubuntu@$MY_EC2" \
  'sudo tail -n 60 /var/log/cloud-init-output.log'
curl -I --max-time 20 "http://$MY_EC2"
set -o pipefail
curl -fsS --max-time 20 "http://$MY_EC2" |
  grep -iF 'Hey me!' && echo YES
echo "Exit code: $?"
terraform plan
ssh -o IdentitiesOnly=yes -i ~/.ssh/id_rsa_level3 \
  "ubuntu@$MY_EC2" \
  'sudo ansible-playbook -i localhost, /opt/config-and-deploy.web.yml'
```

Removal

```
terraform plan -destroy -out=destroy.tfplan
terraform apply destroy.tfplan
aws ec2 describe-instances --region eu-north-1 \
  --instance-ids i-0037b3fe4cc3dfff8 \
  --query 'Reservations[].Instances[].{InstanceId:InstanceId,State:State.Name}' \
  --output table --no-cli-pager
```